SHAHID AFRIDI

Upgrading the Linked List

Doubly and Circular Linked Lists

SHAHID AFRIDI

The Problem with our Basic Linked List

A standard (Singly) Linked List is a **one-way street**. Each node only knows its next neighbor. You can't go backward without starting over from the head.

What if we need to move freely, forward AND backward ?

01

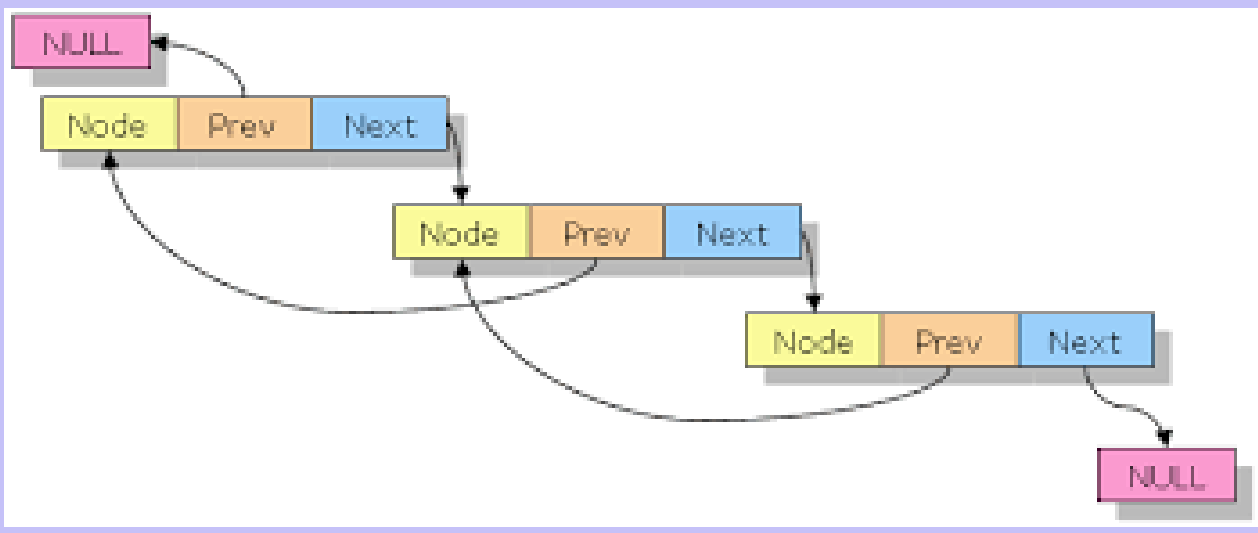
SHAHID AFRIDI

The Solution: A Doubly Linked List

Your Browser History. The "**Back**" button is a previous pointer. The "**Forward**" button is a next pointer. You can navigate in both directions with ease.

How it Works

- Each node now stores **three** pieces of information



its data, a to the next node, and a pointer to the previous node.

SHAHID AFRIDI

The Upgraded Building Block

We just need to add one more property to our **Node** class: self.prev.

```python
# The blueprint for our two-way link
class DoublyNode:
    def __init__(self, data):
        self.data = data
        self.next = None
        self.prev = None # The new superpower!

# A <-> B <-> C
```

SHAHID AFRIDI

Deleting a Node is Now Trivial

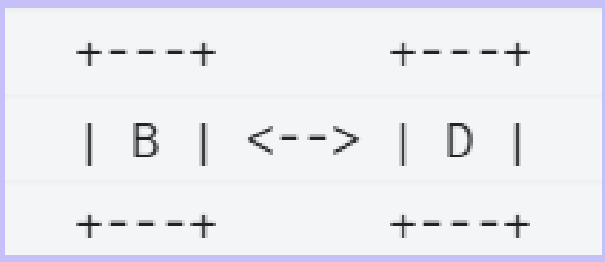
To remove node C from the chain B <-> C <-> D

1. Go to node **B**. Change its **next** pointer to point to **D**.
2. Go to node **D**. Change its **prev** pointer to point to **B**.

- That's it! C is now completely bypassed. With a singly linked list, we wouldn't have had an easy way to access B from C.

"before" chain

"After" chain



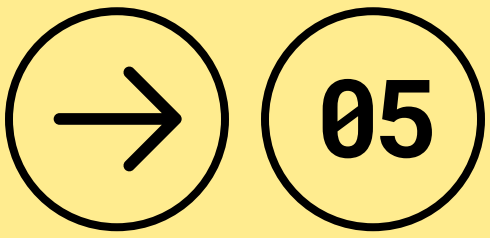
SHAHID AFRIDI

The Circular Linked List

A music playlist on repeat, or a carousel/merry-go-round. When you get to the last item, the "**next**" one is the very first item again. There is no **None** at the end.

How it Works

- The **next** pointer of the very last node (the **tail**) points back to the **head** of the list.



SHAHID AFRIDI

When is an Endless Loop Useful?

- **Use Case 1: Round-Robin Scheduling.** In operating systems, tasks can be put in a circular list. The CPU gives a time slice to each task and then moves to the next, cycling through them continuously.
- **Use Case 2: Photo Galleries.** When you're viewing photos, clicking "next" on the last photo takes you back to the first one.

The Danger: Be careful when traversing! You need to have a clear stopping condition (e.g., stopping when you reach the head again), otherwise you'll create an infinite loop in your code.

SHAHID AFRIDI

A simple summary table

Type	Key Feature	Pro	Con
Singly	**next** pointer only	Simple, less memory.	Can't go backward.
Doubly	**next** and **prev**	Easy two-way travel.	More memory per node.
Circular	**tail.next** = head	Great for cycling.	Traversal can be tricky.

Thank You So Much

Follow me to get more information

share this post with others who might find it helpful!